



Film & Kino Agent

Sitzung 04 – Tool Use & Agenten

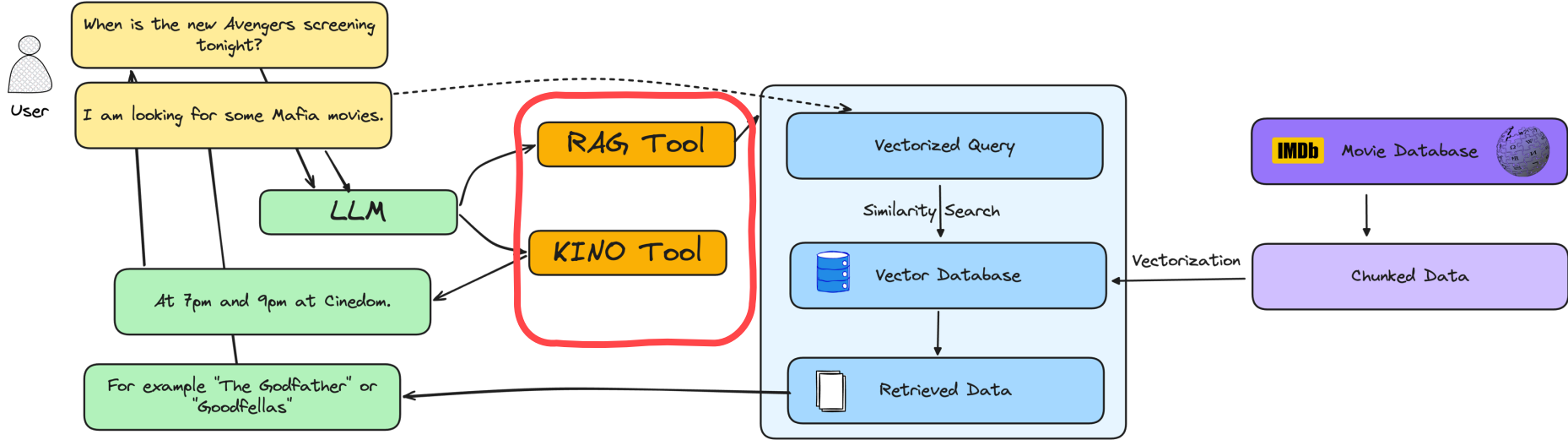
Prof. Dr. Richard Sieg
SoSe 2026

DIS22a

Termine

Datum	Inhalt
15.4	Kick-Off, Projektvorstellung, Setup
22.4	UI bauen, LLM-API-Anbindung, Prompt Engineering
29.4	Embeddings & Vektordatenbanken
6.5	Filmdatenbank vektorisieren
13.5	<i>Home Office</i>
20.5	RAG: Theorie + Implementation (Projektwoche)
27.5	<i>Home Office</i>
3.6	Tool Use & Agenten
10.6	Agenten-Implementation + Theorie: Evaluation
17.6	Gegenseitiger Nutzertest + Feinschliff
24.6	Feinschliff, Bugfixes
1.7	Abschlusspräsentationen

Architektur



Tool Use

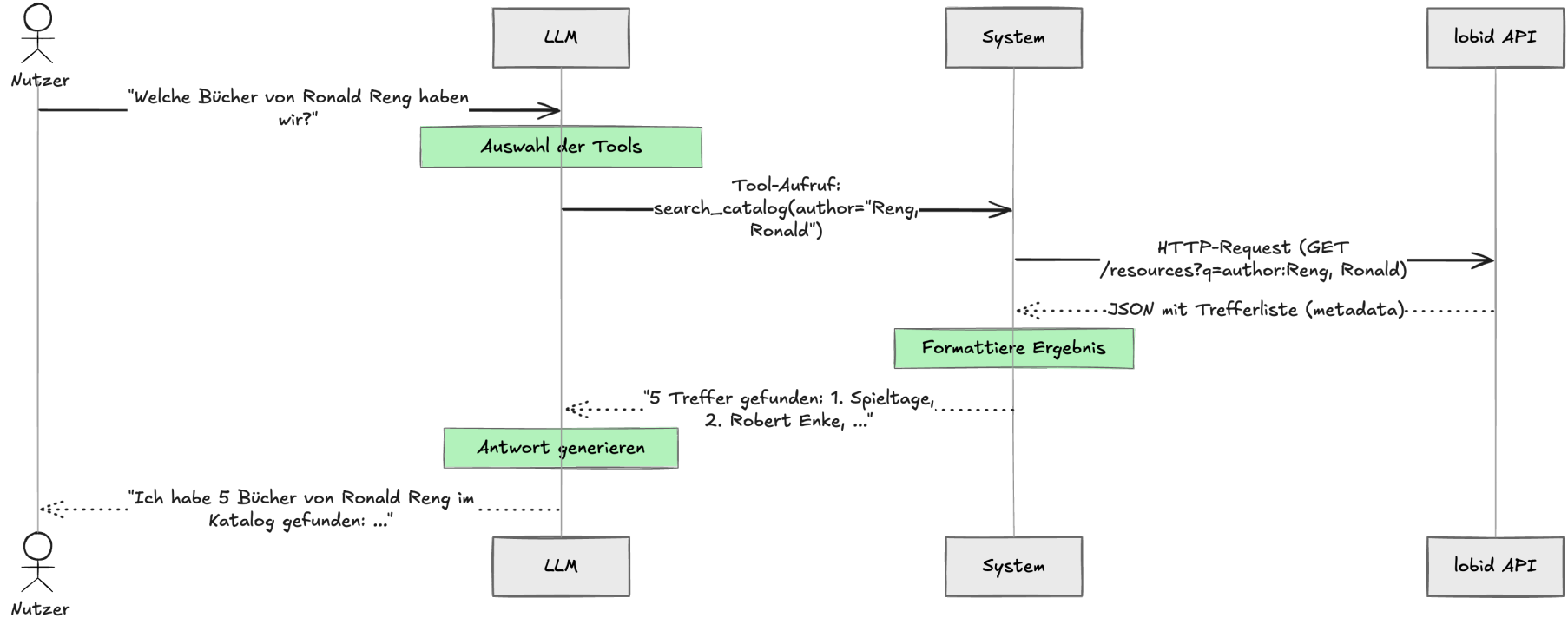
Das LLM ruft strukturiert externe Funktionen auf, statt nur Text zu erzeugen.

- **Definition:** Entwickler beschreibt Tools mit Name, Beschreibung, Parameter-Schema
- **Auswahl:** LLM wählt ob und welches Tool, erzeugt die Argumente
- **Ausführung:** App ruft das Tool auf, gibt das Ergebnis zurück ans LLM

Zentral dabei: Das LLM führt nichts selbst aus.

- LLM erzeugt nur den Aufruf (strukturiertes JSON)
- Die Anwendung führt aus
- LLM sieht das Ergebnis und formuliert die Antwort

Tool Use



Tool Use: Filmbeispiel

```
{  
  "name": "search_movies_rag",  
  "description": "Semantische Suche in der  
Filmdatenbank",  
  "parameters": {  
    "query": "Plot-Beschreibung des gesuchten Films",  
    "genre": "optionaler Genre-Filter",  
    "year_from": "optionales Startjahr"  
  }  
}
```

Empfehl mir Mafia-Dramen aus den 2000ern

```
{  
  "tool": "search_movies_rag",  
  "args": {  
    "query": "powerful crime family struggles for control",  
    "genre": "Crime",  
    "year_from": 2000  
  }  
}
```

Was ist ein Agent?



Simon Willison

I think “agent” may finally have a widely enough agreed upon definition to be useful jargon now

I’ve noticed something interesting over the past few weeks: I’ve started using the term “agent” in conversations where I don’t feel the need to then define it, roll my eyes or wrap it in scare quotes.

This is a big piece of personal character development for me!

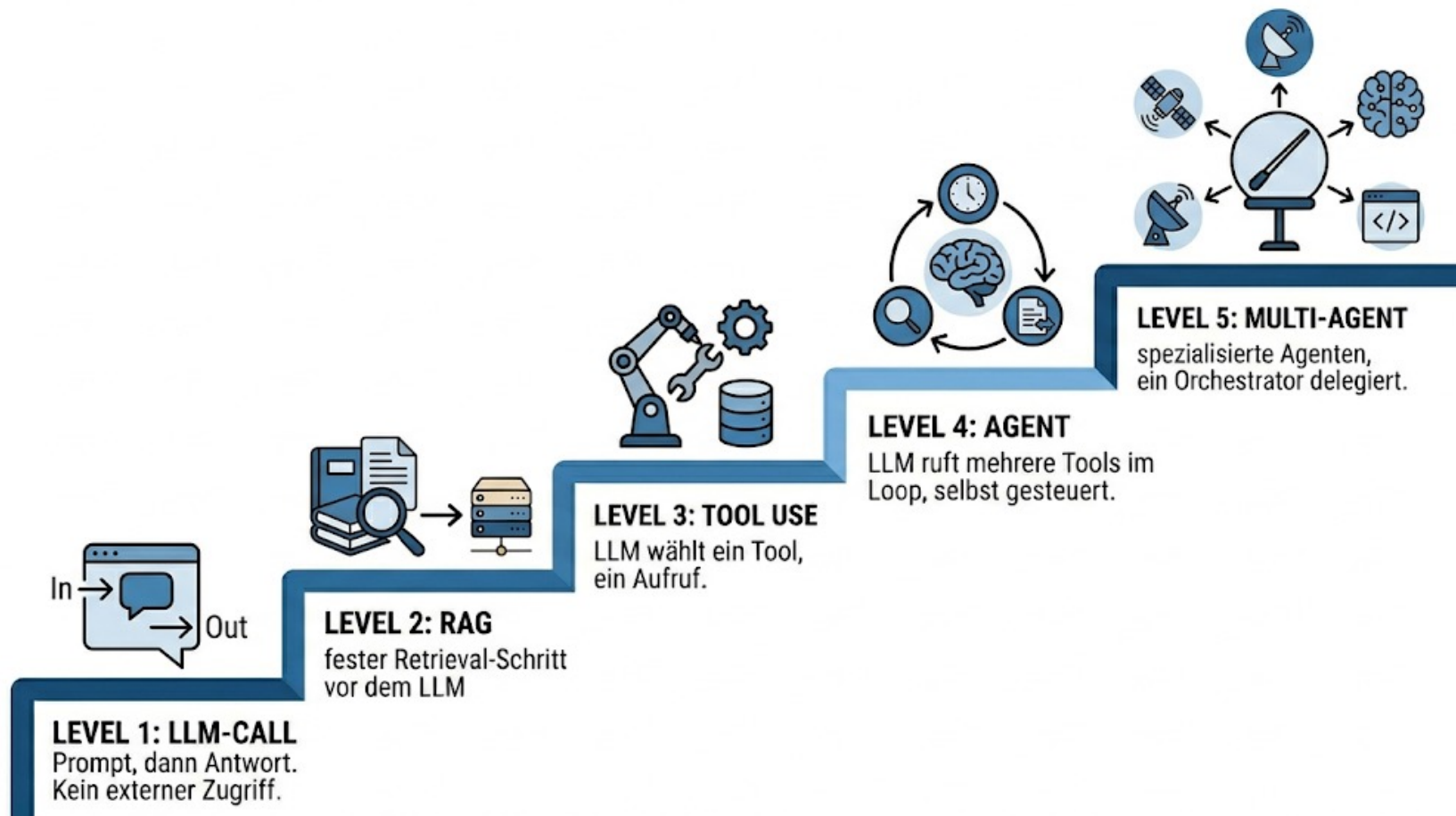
Moving forward, when I talk about agents I’m going to use this:

An LLM agent runs tools in a loop to achieve a goal.

Drei Schlüsselwörter:

- **Tools:** Zugriff auf APIs, Datenbanken, Code-Ausführung
- **In a loop:** LLM wählt selbst das nächste Tool, wertet aus, entscheidet weiter oder fertig
- **Goal:** arbeitet auf ein Ziel hin, nicht auf eine einzelne Antwort

Stufen der LLM Nutzung



ReAct Framework

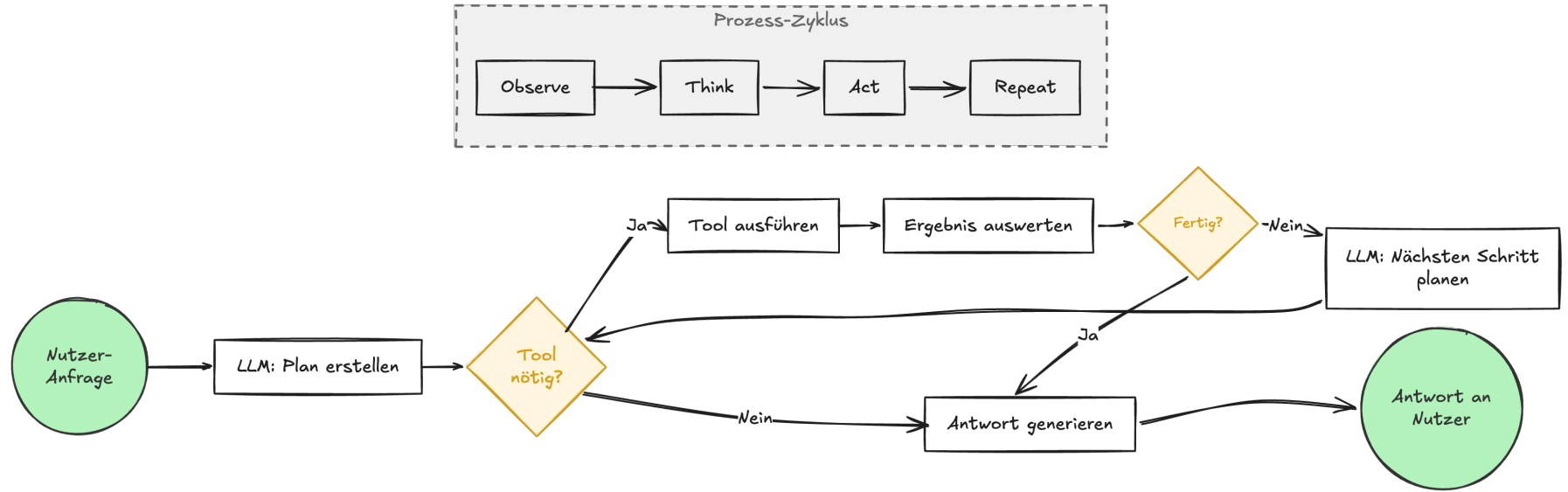
- Kernidee: Abwechselnd Denken (Reasoning) und Handeln (Acting).
- **ReAct (Reason + Act):** Das LLM wechselt zwischen Nachdenken und Handeln. Es schreibt seine Überlegungen auf, entscheidet sich für eine Aktion, beobachtet das Ergebnis und überlegt dann weiter.

```
Thought:    Welche Mafia-Filme gab es in den 2000ern?
Action:     search_movies_rag("crime family power struggle",
                             genre="Crime", year_from=2000)
Observation: 1. The Departed (2006), Rating 8.5 ...

Thought:    Läuft davon etwas im Kino?
Action:     get_showtimes(movie="The Departed")
Observation: Keine aktuellen Vorstellungen.

Thought:    Empfehlung ohne Kino-Info geben.
Action:     finish("Hier sind meine Empfehlungen: ...")
```

Agenten Prozess



Tools bauen: Pydantic AI

```
from pydantic_ai import Agent, RunContext
from dataclasses import dataclass

@dataclass
class AgentDeps:
    qdrant: QdrantClient
    tmdb_api_key: str

agent = Agent("gemini-2.5-flash", deps_type=AgentDeps,
             system_prompt="Du bist CineBot, ein Film-Assistent.")

@agent.tool
async def search_movies_rag(
    ctx: RunContext[AgentDeps],
    query: str, genre: str | None = None,
) -> str:
    """Semantische Suche in der Filmdatenbank.
    Args:
        query: Englische Plot-Beschreibung
        genre: Optionaler Genre-Filter
    """
    ...
```

Dependency Injection:
RunContext liefert
Clients und Keys, keine
globalen Variablen

Type Hints sind
Schemas: Pydantic
erzeugt das JSON-
Schema automatisch

Docstrings sind Prompts:
das LLM liest sie zur
Tool-Auswahl

Agent-Frameworks im Überblick

Provider-native SDKs (auf ein Modell optimiert)



OpenAI Agents SDK

- Swarm-Nachfolger
- Sandboxing
- Codex-Integration

Swarm 



Claude Agent SDK

- Computer Use als Primitive
- autonomes Coding



Google ADK

- hierarchischer Agent-Baum
- A2A-Protokoll

Unabhängige Frameworks (modell-agnostisch)



Pydantic AI

- typsicher FastAPI-Style schlank



LangGraph

- graphbasiert Checkpoints
- Enterprise-Standard



AutoGen / MS Agent Framework

- Multi-Agent, Azure-nativ



CrewAI

- rollenbasierte Multi-Agent-Systeme



smolagents (Hugging Face)

- minimalistisch, smolagents
- Code-First

Open-Source Agenten

OpenClaw (Peter Steinberger, Jan 2026, Open Source)

- Führt Aufgaben aus: Dateien, E-Mails, APIs, Browser
- Architektur: Message Gateway, Agent, Skills, Memory
- NVIDIA baute NemoClaw, Gründer ging zu OpenAI
- ⚠ Remote Code Execution gefunden, ein Lethal-Trifecta-Lehrstück (siehe später)

Hermes Agent (Nous Research, Feb 2026, Open Source, MIT)

- Lokal auf eigener Hardware, persistentes Memory
- Self-Improvement Loop, 40+ Tools, MCP
- Multi-Plattform: Telegram, Discord, Slack, Signal, ...

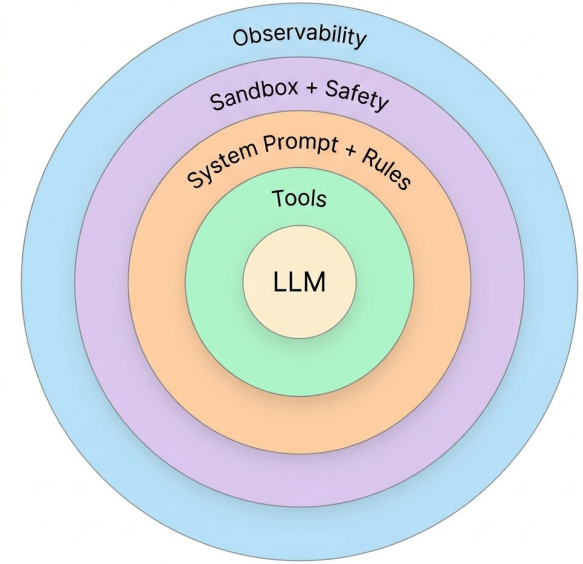
Pi (Earendil Inc., Open Source, MIT)

- Minimaler terminal-basierter Coding Harness

Harness Engineering

Harness: alles, was den Agenten umgibt und steuert.

- System Prompt: Instruktionen, Persona, Grenzen
- AGENTS.md / Rules: projektspezifische Regeln (für Coding/Task Agents)
- Tool-Definitionen: verfügbare Werkzeuge und Parameter
- Sandbox / Safety: erlaubte Aktionen (lesen ja, löschen nein)
- Memory / Kontext: was bleibt über Schritte erhalten
- Observability: Logging, Tracing, was tat der Agent warum

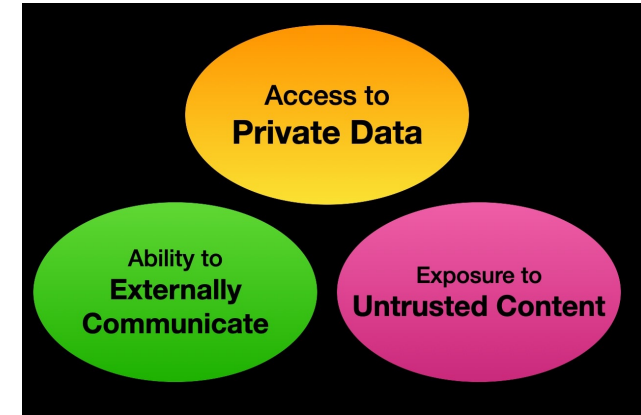


Sicherheit: Lethal Trifecta

Simon Willison (2025): drei Eigenschaften, die zusammen Prompt Injection gefährlich machen.

- **Private Daten:** E-Mails, Datenbanken, Dateien
- **Untrusted Input:** Webseiten, E-Mails, Nutzereingaben
- **External Actions:** senden, APIs aufrufen, schreiben

Alle drei zusammen: Angreifer kann per manipuliertem Inhalt private Daten exfiltrieren.



Lethal Trifecta



Code Wall